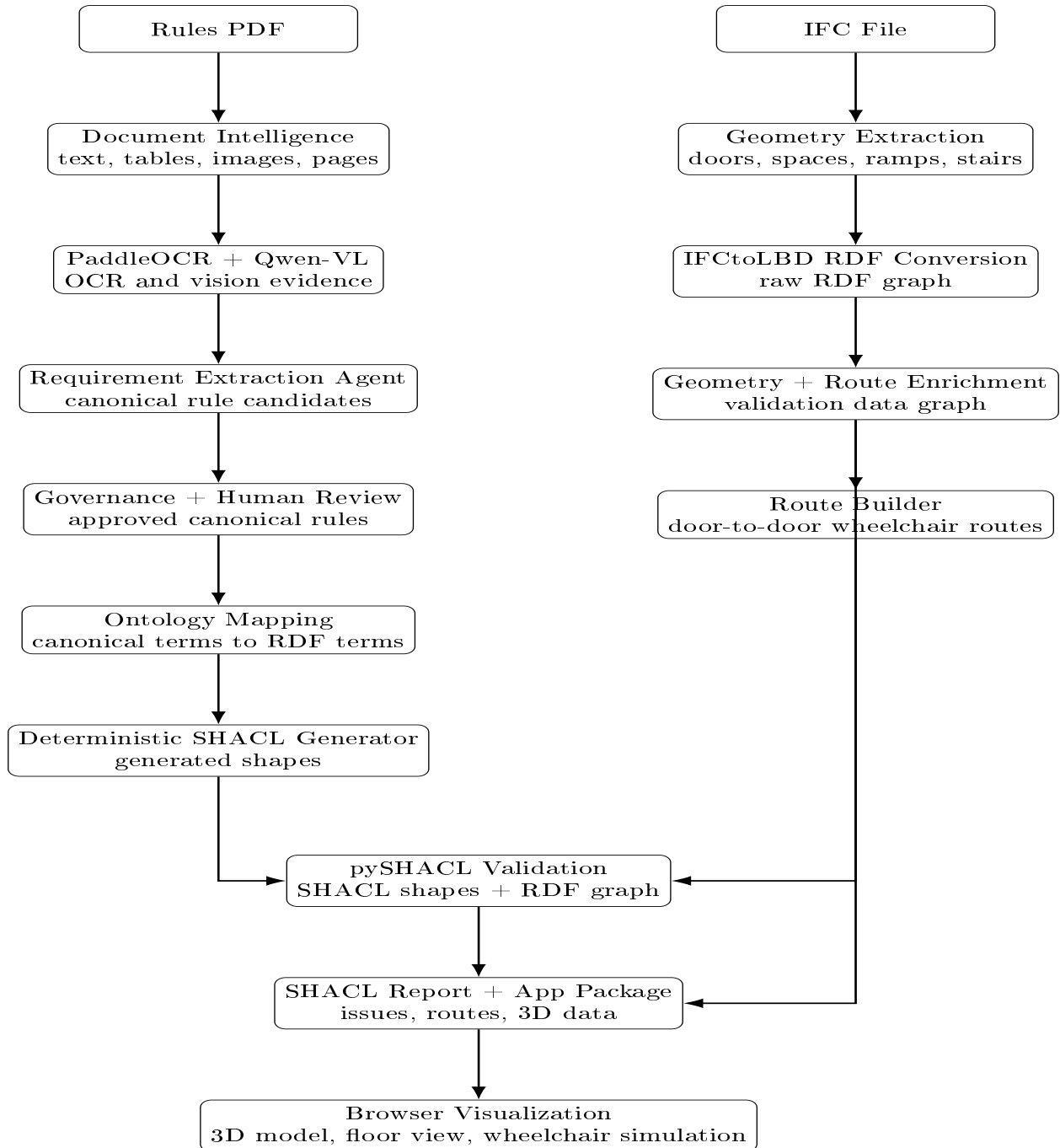


PDF and IFC Workflow Report

Mid Term 1 Accessibility Checking Prototype

12 June 2026

1 Flow Diagram



2 Short Explanation

The system receives two files:

- **IFC file:** the building model, for example `AC20-Institute-Var-2.ifc`.
- **Rules PDF:** the rule book or standard, for example `planungsgrundlagen_barrierefreies_bauen.pdf`.

The two files move through separate pipelines first. The PDF becomes approved canonical rules and then SHACL constraints. The IFC becomes an RDF graph with geometry and route data. Both sides meet at pySHACL validation.

3 Step-by-Step File Journey

Step	Component	File or data entering	What happens
1	<code>preprocess.py</code>	IFC file and rules PDF	This is the main runner. It sends the IFC into the building workflow and the PDF into the rule extraction workflow. Output is written to <code>output/app_package/</code> .
2	Geometry extraction	IFC file	The system reads doors, spaces, walls, stairs, ramps, slabs, and columns. It extracts positions, bounding boxes, widths, heights, floor levels, labels, and IFC IDs.
3	IFCtoLBD	IFC file	The IFC model is converted into RDF. RDF stores facts as graph triples, for example that one object is an <code>IfcDoor</code> . Output: <code>raw_lbd_graph.ttl</code> .
4	RDF enrichment	Raw RDF graph and geometry objects	The system adds measured values to the RDF graph, such as door width, ramp slope, clear route width, and turning space. Output: <code>lbd_graph.ttl</code> .
5	Route builder	IFC geometry and spaces	The system builds wheelchair route edges, such as door-to-door paths. It records route width, door width, turning space, ramp data, and stair intersections.
6	3D package writer	Elements, routes, and issues	The system prepares browser files such as <code>app_data.json</code> , <code>route_model.glb</code> , and <code>route_graph.bin</code> .
7	Document Intelligence	Rules PDF	The PDF is split into text chunks, table chunks, image chunks, and page metadata. It also renders full pages and table regions as images, so vector diagrams and page-drawn tables can be seen.
8	PaddleOCR	Embedded images, rendered pages, rendered table regions	OCR means text recognition from images. PaddleOCR reads visible text from image evidence and attaches the result to each image chunk.

9	Image ranking	All image chunks with OCR and page context	Every image chunk is scored using general evidence signals: measurement density, numeric density, normative language, OCR text quality, page context, table context, visual size, and uniqueness.
10	Vision model	Top-ranked image chunks	Qwen-VL looks at selected images and writes observations about visible measurements, diagrams, labels, tables, limits, and ranges.
11	Requirement Extraction Agent	Text, tables, OCR text, and vision observations	The agent extracts canonical rule candidates. It must output neutral rule objects only. It is not allowed to output SHACL, RDF, or ontology mappings.
12	Candidate governance	Canonical rule candidates	The system compares candidates, detects duplicates, checks conflicts, and chooses the best-supported candidate using evidence quality.
13	Human review	Selected rules and mapping candidates	Rules cannot become SHACL automatically. A reviewer, timestamp, rationale, and decision must exist before publication.
14	Ontology mapping	Approved canonical rules and RDF graph	The system maps neutral rule terms to RDF properties. For example, a canonical ramp-width rule maps to the RDF property that stores ramp usable width.
15	SHACL generator	Approved rules and approved mappings	Python templates generate SHACL deterministically. No LLM generates SHACL. Output: <code>generated_accessibility_rules.shacl.ttl</code> .
16	pySHACL validation	Generated SHACL and enriched RDF graph	pySHACL checks whether the building graph satisfies the generated SHACL constraints. Output: <code>shacl_report.ttl</code> .
17	Issue conversion	SHACL report, elements, and routes	SHACL validation results are converted into app issue records, such as route too narrow, ramp too steep, or turning space too small.
18	Browser frontend	App package files	The browser shows the 3D model, floors, routes, wheelchair simulation, issue list, and SHACL report data. The browser is only a viewer. Compliance is decided by pySHACL.

4 PDF Pipeline In Simple Form

Rules PDF

- > Document Intelligence
- > text chunks, table chunks, embedded images, rendered full pages, rendered table regions

- > PaddleOCR
- > image ranking
- > Qwen-VL vision
- > requirement extraction agent
- > canonical rule candidates
- > governance selection
- > human review
- > ontology mapping
- > deterministic SHACL generation
- > generated SHACL file

5 IFC Pipeline In Simple Form

IFC file

- > geometry extraction
- > IFCtoLBD RDF conversion
- > geometry and route enrichment
- > route graph generation
- > validation data graph

6 Where Both Pipelines Meet

Generated SHACL file + validation data graph

- > pySHACL
- > SHACL report
- > app_data.json
- > browser visualization and wheelchair simulation

7 Important Notes

- The PDF pipeline now sees more than embedded images. It also sees rendered pages and rendered table regions.
- This helps when a PDF contains diagrams, table lines, arrows, or measurements that are drawn directly on the page.
- The SHACL generator is deterministic. It uses Python templates, not an LLM.
- The browser does not decide compliance. It only displays the prepared result.
- The system is stronger now, but it is not production-perfect. In the latest audit, `door_width` was still missing from the final canonical rules.

8 Main Output Files

File	Purpose
output/app_package/document_chunks.json	Structured PDF output: pages, text, tables, image evidence, OCR text, vision text, and ranking metadata.
output/app_package/canonical_rules.json	Extracted canonical rules from the PDF.
output/app_package/rule_extraction_audit.json	Evidence, model calls, accepted outputs, rejected outputs, and retrieval audit.
output/app_package/rule_review.json	Human review records for rules and mappings.

output/app_package/ontology_mappings.json	Mapping from canonical rule terms to RDF classes and properties.
output/app_package/generated_accessibility_rules.shacl.ttl	Generated deterministic SHACL shapes.
output/app_package/lbd_graph.ttl	RDF graph used as validation data.
output/app_package/shacl_report.ttl	pySHACL validation report.
output/app_package/app_data.json	Browser data: building elements, routes, issues, and simulation data.
output/app_package/route_model.glb	3D model used by the browser.